

Introduzione

La sicurezza delle comunicazioni e delle reti informatiche rappresenta oggi una delle sfide più critiche data la continua evoluzione delle tecnologie e il costante aumento nella complessità dei sistemi. Questi fattori portano alla scoperta di nuove vulnerabilità e vettori di attacco. In particolare ci focalizzeremo sui Covert Channel, una minaccia particolarmente insidiosa, in quanto permette la trasmissione di informazioni fra due entità ma la cui individuazione risulta complessa.

Nella seguente presentazione, verranno indicate le fasi di analisi e sviluppo di un Covert Channel basato sul protocollo ICMP.

ICMP

ICMP è un protocollo che opera al livello di rete (livello 3 modello ISO/OSI), e permette la segnalazione di errori, la diagnostica della rete e la messaggistica di controllo. Queste caratteristiche lo rendono fondamentale nelle reti e proprio per questo, non può essere completamente disabilitato o filtrato senza compromettere l'operatività della rete stessa. Ciò lo rende un candidato ideale per la creazione di canali di comunicazione nascosti.

Covert Channel

Un Covert Channel è un attacco che permette (in ambienti ritenuti sicuri) la capacità di comunicare o trasferire dati in maniera non autorizzata e non voluta. Solitamente sfruttano vulnerabilità o comportamenti non previsti nei sistemi per operare al di fuori dei soliti meccanismi di comunicazione. Ciò gli permette di non generare segnali di un uso improprio del sistema il che li rende difficili da rilevare.

Per poter essere efficace, un Covert Channel deve possedere determinate caratteristiche.

- Furtività: evitare di attirare le attenzioni sia degli amministratori che degli strumenti di sicurezza.
- Capacità di trasmissione: la quantità di dati che il canale trasmette in un determinato intervallo di tempo. Una quantità elevata può portare ad una rilevazione del canale.
- Indistinguibilità: la trasmissione mantiene inalterato il funzionamento delle risorse utilizzate. Ovvero il Covert Channel si rende indistinguibile dalla risorsa autorizzata.

Tipologie di Covert Channel

Timing Covert Channel

Permettono ad un osservatore (che può essere una persona od un processo) di acquisire informazioni attraverso il cambiamento del tempo di risposta di una risorsa.

Storage Covert Channel

Il canale funziona scrivendo dei dati su un'area di memoria condivisa accessibile da tutte le entità coinvolte nella comunicazione.

Behavioural Covert Channel

Le informazioni vengono codificate mediante la variazione del comportamento del sistema. Quindi i dati vengono ricavati tramite l'osservazione del sistema piuttosto che attraverso l'accesso alla memoria.

Struttura Comunicazione

ATTACCANTE

- **Path file:** Percorso per il file di configurazione da caricare. In questo file JSON viene specificato l'indirizzo IP della vittima, la metodologia di attacco e la lista dei proxy che si vogliono usare.

Tramite la linea di comando, che ha lanciato l'esecuzione dello script, si ricava il valore del path dove è presente il file di configurazione dell'attaccante. In esso viene definito l'indirizzo IP della vittima, il metodo di attacco e i proxy che si vogliono utilizzare per l'esfiltrazione dei dati.

Procede poi a verificare quali proxy (fra quelli presenti) siano utilizzabili; ovvero quali riescono a comunicare sia con l'attaccante che con la vittima. Dopodiché inserirà il comando che si vorrà far eseguire dalla vittima o il dato che gli si vuole mandare. Nel caso non si abbia una connessione diretta con la vittima, ma si utilizzano dei proxy come intermediari, si aspetterà che essi inoltrino i dati che la vittima ha inviato a ciascuno di essi. Una volta ricevuti tutti i dati dai proxy, l'attaccante li riordinerà per ricavare il messaggio correttamente e potrà poi decidere se inviare un ulteriore comando o se terminare la comunicazione. In quest'ultimo caso, notificherà l'evento a ciascuno dei proxy.

PROXY

- **Indirizzo IP:** Rappresenta l'indirizzo IP dell'attaccante. Il proxy accetterà solo le connessioni che combaciano con questo indirizzo; qualunque altra connessione verrà rifiutata.

Durante la fase di inizializzazione il proxy necessita di sapere l'indirizzo IP dell'attaccante così da poter stabilire una connessione solamente con lui; tutti i tentativi di connessioni provenienti da indirizzi diversi verranno ignorati e scartati.

Stabilita una comunicazione con esso, il proxy aspetterà un messaggio indicante l'indirizzo IP della vittima oltre alla tipologia di Covert Channel scelto. Ricevuto l'indirizzo il proxy si assicurerà che la comunicazione con esso sia possibile. Nel caso non sia possibile, il proxy aggiornerà l'attaccante che non potrà essere utilizzato per l'esfiltrazione dei dati dalla vittima. Entrambe le entità procederanno poi alla chiusura della comunicazione.

I successivi messaggi da parte dell'attaccante potranno essere:

- Il messaggio indicante il comando che il proxy dovrà inoltrare alla vittima
- Il messaggio indicante che il proxy non ha ricevuto il comando e che dovrà procedere ad ascoltare i dati provenienti dalla vittima

- Il messaggio indicante la terminazione della comunicazione con l'attaccante.

I primi due messaggi servono siccome si potranno usare molteplici proxy ma solo di loro riceverà il comando da inoltrare .

VITITMA

- Numero dei Proxy: Numero minimo di connesisoni necessari per l'esecuzione dell'attacco.

Nella fase di inizializzazione alla vittima verranno indicate la quantità di connessioni attive necessarie per l'esecuzione dell'attacco. Queste connessioni verranno usate sia per la ricezione del comando sia per l'esfiltrazione dei dati. Nel caso il requisito non venga soddisfatto la vittima procederà a terminare il programma.

Quindi la vittima rimarrà in attesa delle connessioni. Durante questa fase, il proxy (o l'attaccante) che si vuole connettere indicherà nel messaggio usato per connettersi, l'attacco che si userà per lo scambio dei dati.

Definiti i porxy disponibili, la vittima aspetterà di ricevere un comando. Alla sua ricezione, lo eseguirà ed i dati ricavati verranno inviati all'attaccante tramite le connessioni precedentemente stabilite. Nel caso siano presenti molteplici proxy connessi, si definirà una lista indicante i dati che ciascuno di essi dovrà ricevere. L'ultimo messaggio che verrà inviato a ciascuno dei proxy sarà quello che indica che tutti i dati sono stati mandati.

Implementazione

Si ritiene fondamentale la presenza di più proxy intermediari fra l'attaccante e la vittima. Ciò permette di distribuire in modo omogeneo il carico sulla rete che viene generato dall'esfiltrazione dei dati. Se invece si usasse un singolo proxy, tutto il flusso verrà indirizzato verso un'unica destinazione.

Un limite alla quantità di dati inviabili in un intervallo di tempo ed un periodo di riposo prima di poterne inviare degli altri. Questo per permettere l'invio di file di grosse dimensioni evitando di congestionare il canale di comunicazione.

L'invio di un messaggio di tipo Request richiede la ricezione di una risposta; che normalmente sarà uguale al messaggio inviato. Una soluzione può essere l'invio di soli messaggi di tipo Reply oppure la disabilitazione delle risposte così da poterne inviare una noi stessi.

IMPLEMENTAZIONE TIMING COVERT CHANNEL

Per poter temporizzare i dati da inviare, c'è bisogno di definire un metodo che permetta la codifica e decodifica degli intervalli temporali.

- Nella `\textbf{codifica}` si calcolerà il delay temporale che verrà associato al dato da inviare. Esso restituirà il tempo da aspettare prima di inviare un pacchetto. Per esempio al dato binario 1001 potrebbe essere associato il tempo 3; quindi il programma, prima di mandare un nuovo pacchetto, aspetterà tre secondi.

- Per la \textbf{decodifica}, il destinatario osserva il flusso dei pacchetti per poter calcolare la differenza di tempo fra il pacchetto precedente e quello attuale. Il dato verrà quindi ricavato controllando a quale informazione è stato mappato l'intervallo ricevuto. Per esempio se l'intervallo di tempo risultasse di 6 secondi, il destinatario controllerà a chi è associato tale tempo e ricaverà il dato binario 1101.

IMPLEMENTAZIONE DELLA FUNZIONE

In una prima implementazione si ricava il tempo di delay associando ad ogni valore una distanza temporale basata sulla distanze che due dati devono avere fra di loro.

$$\text{tempo_base} + \text{indice} * \text{distanza_tempi}$$

Lo svantaggio dell'approccio sono i tempi che si aspettano. Per risolvere il problema, si è pensato di ridurre i valori inviabili solo ai caratteri stampabili che nella codifica ASCII vanno da 32 a 126. Tuttavia questo lega il metodo ad una codifica specifica inoltre avrebbe portato alla perdita di tanti se non si fosse esfiltrato un documento.

Sviluppando meglio l'idea e prendendo spunto da ICMP Exfill, implementazione, il tempo di delay ora viene visto come il valore intero del byte che si vuole inviare. Ciò è possibile siccome si lavora con una sequenza di bit. Dopodichè si sono normalizzerà l'intervallo dei valori ottenibili intorno ad un valore minimo ed uno massimo.

$$\text{min_delay} + (\text{byte}/255) * (\text{max_delay} - \text{min_delay}).$$

Tramite questo approccio, il range dei delay viene ristretto ad un intervallo preciso e definito. Inoltre permetterà di regolare la velocità di trasmissione delle informazioni (che può essere maggiore o minore) sacrificando la sensibilità della comunicazione ai ritardi che possono avvenire durante il transito dei pacchetti.

Dall'implementazione precedente, si note che un invio di pacchetti con pattern costanti può essere facilmente individuato dai sistemi di sicurezza. Una funzione in cui lo schema dei tempi di invio dei pacchetti sembri casuale, permetterà di essere rilevati con maggiore difficoltà. Si aggiungerà quindi al Timing Covert Channel del rumore così che i tempi di delay sembrino casuali. Segue che il tempo di delay associato a un byte non sarà più costante ma spazierà fra un range di valori, nel quale potrà variare.

Questo porta che, per potersi scambiare i dati, il mittente e il destinatario devono potersi sincronizzare sulla quantità di rumore che si è aggiunto. Le strade possibili sono due:

- L'entità mittente creerà un numero randomico e lo usa per calcolare il rumore. Le entità si dovranno quindi sincronizzare sul seed usato oltre al numero di estrazioni effettuate. Se i valori estratti non combaciassero; le entità non riusciranno a scambiarsi i dati correttamente.
- Il mittente indica il rumore generato nel pacchetto stesso (per esempio in uno dei campi o il payload). Il ricevente otterrà il valore del rumore generato dalla risorsa condivisa (il

pacchetto). In questo caso non si avrà un Timing Channel puro, ma un Timing Channel ibrido.

- Gli intervalli temporali non trasportano alcun dato. Ma in questo caso non si tratterà più di un Timing Covert Channel.

Siccome il destinatario rimane in ascolto e, all'arrivo di un pacchetto, ricava il dato associato all'intervallo di tempo; avrà bisogno di un messaggio che indichi quando la trasmissione inizia. Nel timing channel si dovrà quindi prevedere un modo per indicare l'inizio della comunicazione. Il metodo potrà essere una particolare sequenza inserita in un campo del pacchetto mentre oppura una particolare tipologia di messaggio ICMP (se non entrambi i metodi).

IMPLEMENTAZIONE STORAGE COVERT CHANNEL

In questo Storage Covert Channel la risorsa condivisa sarà il pacchetto ICMP inviato. Il mittente memorizzerà i dati da esfiltrare nei campi presenti nel messaggio mentre il destinatario, una volta ricevuto, leggerà i valori codificati all'interno dei pacchetti.

Nelle Tabelle sono indicate le tipologie di messaggi sfruttati, il minimo di byte che verranno inviati a pacchetto e quali campi sono stati utilizzati. Nel caso di alcune tipologie (come i messaggi Echo Request/Reply), si avranno delle variazioni sia nella quantità di dati esfiltrabili sia sulla quantità di byte che vengono trasmessi per datagram. Ciò è dovuto alla presenza del campo data che permette di inserire una quantità maggiore di dati.

Nelle prossime slides esamineremo come lo Storage Covert Channel inserirà i dati nei campi.

INSERIMENTO DEI DATI NEL CAMPO HEADER+64 BIT

Nel campo sono presenti l'header IP più 8 byte per una lunghezza complessiva di 28 byte. Nel nostro caso rappresenterà un pacchetto ICMP/IP; sebbene si possono usare anche altre combinazioni di protocolli (e.g TCP/IP). E siccome non rappresenta un datagram da inviare ma già spedito, si possono sfruttare un maggior numero di campi. In questo campo si potranno quindi inserire 9 byte di dati.

HEADER IPv4

- Time to live & 1 byte & Tempo di vita del pacchetto
- Total length & 2 byte & Dimensione dell'intero pacchetto
- Identification & 2 byte & Identifica i frammenti di un pacchetto IP

Header ICMPv4

- Identifier & 2 byte & Identificativo del messaggio di richiesta invocato
- Sequence & 2 byte & Numero di sequenza del messaggio di richiesta invocato

INSERIMENTO DEI DATI NEL CAMPO UNUSED

Dalle specifiche RFC (792 e 4434) il campo deve essere vuoto. Quindi utilizzando i metodi presenti nel framework Scapy per la creazione del pacchetto, qualsiasi valore che si voglia inserire al suo interno verrà azzerato. Il problema viene risolto tramite l'uso della libreria struct. Con essa è possibile costruire un pacchetto avente i dati all'interno del campo e successivamente si userà Scapy solo per l'invio. Siccome il suo utilizzo rende il messaggio non conforme agli standard RFC, un IDS che implementi la Deep Packet Inspection rileverà l'anomalia.

INSERIMENTO DEI DATI NEL CAMPO DATA

Nel campo il mittente può inserire quanti dati preferisce. Ma per sicurezza la dimensione sarà o 32 o 64 byte. Inoltre una quantità di dati eccessiva può far mandare più pacchetti di risposta nel caso si fosse inviata una richiesta.

INSERIMENTO DEI DATI NEL CAMPO TIMESTAMP

Contiene i millisecondi dalla mezzanotte UT e ciascun campo ha un ordine temporale specifico; che dovrà rimanere congruente quando si inseriscono i dati. Il Codice mostra come in ciascun campo timestamp verrà inserito un byte nella parte dei millisecondi.

In ciascun campo timestamp verrà inserito un byte nella parte dei millisecondi. Non è possibile inserire ulteriori informazioni nelle altre parti del campo siccome rappresentano le ore, i minuti ed i secondi. Infatti hanno dei valori definiti e limitati, come si vede (che mostra il valore contenuto nei campi). Un'alternativa è l'inserire il dato in tutto il campo ma, rappresentando un valore temporale, ciò porterebbe al rischio di avere tempi errati o non congruenti. Per esempio il pacchetto viene risultare ricevuto prima che sia stato inviato.

Dalla Figura si può constatare la presenza dell'informazione nella parte dei millisecondi. Dalle informazioni del pacchetto, i caratteri ricavati saranno le lettere R, G ed O; siccome i corrispettivi valori nella codifica ASCII sono 82, 71 e 79.

INSERIMENTO DEI DATI NEL CAMPO POINTER

Indica l'ottetto del messaggio in cui è presente l'errore; tuttavia può contenere dei dati non correlati ad esso. Nel nostro caso, verranno inseriti tanti dati quanto la sua capacità.

INSERIMENTO DATI NEL CAMPO MTU

Nel campo MTU viene indicata la capacità massima del collegamento, Siccome il suo valore non è un valore prestabilito (ed è un intero variabile); l'intera dimensione del campo, potrà essere usato per inserire i dati.

INSERIMENTO DATI NEL CAMPO INVOKING PACKET

Quando si indica il pacchetto che ha causato l'errore, si dovrà stare attenti a non superare la \textit{IPv6 MTU}. Nel nostro caso, ciò non succederà siccome si useranno i protocolli IPv4 la cui intestazione è di 40 byte, ed il protocollo ICMPv4, la cui intestazione è di 8 byte.

Invece nel campo \textit{Invoking Packet} verrà indicato il pacchetto che ha causato l'errore. Nel nostro caso si userà un pacchetto IP/ICMP. Tramite i campi sfruttati si riesce ad inserire \textit{7 byte} di dati.

HEADER IPv6

- Payload Length & 2 byte & Tempo di vita del pacchetto
- Next Header & 1 byte & Indica il tipo di header seguente a quello di IPv6
- Hop Limit & 1 byte & Decrementato di 1 per ogni nodo che inoltra il pacchetto

HEADER ICMPv6

- Identifier & 2 byte & Identificativo del messaggio di richiesta invocato
- Sequence & 2 byte & Numero di sequenza del messaggio di richiesta invocato

IMPLEMENTAZIONE BEHAVIOURAL COVERT CHANNEL

Nel Behavioural Covert Channel implementato, l'evento sarà definito dall'arrivo di un messaggio ICMP. Il destinatario ricaverà il dato trasmesso osservando la tipologia di messaggio ed il codice presente per poi vedere a quale informazione è mappata la coppia. Nella figura si può vedere come le coppie (Tipologia,Codice) sono state mappate ai dati che potranno essere trasmessi. Siccome il numero di eventi possibili sono 17, ognuno permetterà di codificare 4 bit alla volta ($\log_2(16)=4$). Quindi per poter inviare un byte, si dovranno verificare due eventi. L'evento che rimane inutilizzato, verrà usato per indicare l'inizio e la terminazione dell'invio dei dati.

TIPOLOGIE DI MESSAGGI DEPRECATI

Un problema di questa possibile implementazione, sono le tipologie deprecate dei messaggi. Questi messaggi potranno comunque essere inviati ma le linee guida indicano ai router di ignorarli e all'host utente di non mandarli. Inoltre l'uso di tipologie deprecate renderà il canale meno legittimo e maggiormente individuabile siccome questo tipo di traffico sarà visto con sospetto. Infine date le linee guida, non è assicurata la ricezione dei messaggi.

Le tipologie deprecate sono:

- Information Request/Reply è stato deprecato siccome meccanismi come il DHCP, lo hanno sostituito per la configurazione dell'host.
- Source Quench: era concepito come meccanismo per il controllo della congestione. Tuttavia, data la sua inefficienza è stato formalmente deprecato dal 1995. Attualmente viene ignorato siccome è stato sostituito dai meccanismi di controllo della congestione implementati nel protocollo TCP.

TEST E RISULTATI

Inizialmente si sono effettuati test in cui si inviava il dato una singola volta. Dai risultati ottenuti si ottiene che RITA non hanno mostrato la presenza dei Covert Channel siccome i dati sono stati mandati una singola volta nella giornata.

Uno step successivo per l'esecuzione dei test è l'invio degli stessi dati più volte (in particolare tre volte). Inoltre al test si aggiungeranno dei parametri così da poter verificare meglio cosa avviene. Questi parametri saranno:

- La quantità di dati inviati. Possibili dimensioni saranno 1KB, 10KB, 100KB, 1MB.
- I campi usati nei messaggi ICMP dal Covert Channel. Siccome si useranno messaggi ICMP ECHO le variazioni si baseranno sull'utilizzo o meno del campo Identifier o del payload. Nel caso si utilizzasse il payload, quest'ultimo potrà contenere una quantità di dati fissa o variabile per ogni pacchetto.
- Il delay fra i blocchi di dati. I dati verranno suddivisi in blocchi aventi una certa dimensione (e.g 100KB); si può decidere se avere un delay fra l'invio di un blocco e il successivo oppure un invio sequenziale. Nel caso affermativo si invierà un blocco di una certa dimensione e poi si aspetta l'intervallo di tempo. Dopodichè si manda il blocco successivo. L'altro caso è l'invio dei dati sequenzialmente, senza pause.
- Tempo di attesa prima di un nuovo invio dei dati. Una volta che si sono inviati tutti i dati ci potrà essere un tempo di riposo prima di poter inviare nuovamente altri dati. Questo tempo sarà variabile (e.g 1 ora, 30 minuti) e dipenderà dalla quantità di dati esfiltrati (e.g 100KB, 1MB)
- Il mittente del messaggio. Può essere un singolo mittente che invierà tutti i pacchetti oppure si potrà falsificare il campo ed inserire valori diversi; così da far sembrare che il pacco sia stato spedito da un altro host. In questo caso i pacchetti non avranno l'IP della macchina vittima ma un indirizzo preso da un pool di indirizzi IP. Al suo interno ci potranno essere o gli indirizzi IP degli host attualmente attivi sulla rete oppure gli indirizzi IP non utilizzati (e quindi non associati ad alcun host fisico).

L'obiettivo è determinare come RITA risponde alle comunicazioni con queste caratteristiche.

TEST CON MULTIPLI INVII DEI DATI

Principalmente i test avverranno esfiltrando i dati tramite messaggi ICMP Echo nei quali verrà sfruttato il campo payload per l'inserimento delle informazioni.

Dai risultati ottenuti si nota che l'uso di ulteriori mittenti (oltre a quello assegnato all'host), permette di ridurre la gravità assegnata alla connessione e in alcuni casi anche il Beacon score. Nei test in cui si utilizzava solamente l'indirizzo IP assegnato all'host, al canale di comunicazione RITA ha assegnato una gravità media ed un punteggio di Beacon fra il 15% ed il 38%. Al contrario nel test in cui si sono utilizzati più mittenti, la gravità assegnata alla comunicazione è pari a Low mentre il Beacon Score è fra il 13% ed il 16%.

<Test con payload sso, 1MB di dati e un delay durante l'invio >

Dai test risulta anche che un tempo di attesa (abbastanza lungo) fra un'esfiltrazione e la successiva diminuisce la gravità assegnata alla comunicazione (oltre al suo Beacon Score)

<Test con payload sso, 1MB di dati e un delay durante l'invio >

Tuttavia con l'implementazione attuale in alcuni casi il tasso di beaconing peggiora. Siccome per ogni pacchetto si sceglie sempre un mittente diverso (che può combaciare o no con quello prima) e si hanno dei tempi di delay fra un pacchetto e il successivo. Il cambio di mittenti per ogni pacchetto unito ad un tempo di attesa, viene visto da RITA come un impulso di comunicazione. Un'implementazione in cui si sceglie il nuovo mittente dopo aver inviato l'intero blocco di dati, risulterebbe maggiormente efficace nel diminuire la probabilità che il canale venga rilevato.

<Test con payload randomico, 1MB di dati e un delay durante l'invio >

Un invio sequenziale dei dati sembra un metodo efficace per non essere rilevati siccome i test che implementavano la cosa non sono mai stati identificati da RITA. Tuttavia l'efficacia sarà determinata anche in base alla dimensione di dati che si vuole esfiltrare. Una quantità pari ad 1MB (se non più) congestionerebbe il canale di comunicazione con la conseguente creazione di rumore che porterà l'attenzione, da parte degli amministratori e dei sistemi di sicurezza, sul Covert Channel. Invece per ordini di dimensioni minori (sull'ordine dei KB) potrebbe essere una strategia di esfiltrazione utilizzabile.

La precedente affermazione viene consolidata dai test che inviano 100KB di dati. In essi sia se si utilizza il mittente reale o un mittente finto, la comunicazione non viene rilevata da RITA. Per la maggior parte non risulta presente alcun record relativo alla comunicazione, sebbene questo potrebbe essere dato da come Zeek estrae i log dai file pcap e da come RITA elabora questi log. L'unica voce presente è relativa all'uso dei mittenti falsificati con un tempo di attesa pari ad un ora; tuttavia la gravità assegnata è pari a None con un tasso di Beacon del 30% (che potrebbe essere dovuto da come si gestiscono i mittenti).

<Test con payload sso, 100KB di dati e un delay durante l'invio >

<Test con payload randomico, 100KB di dati e un delay durante l'invio>

Un payload variabile non sembra abbia aumentato o diminuito la gravità del canale di comunicazione siccome rispetto al test usante il payload fisso, non risultano forti differenze fra le etichette dei due test.

<Test con payload sso, 1MB di dati e un delay durante l'invio >

<Test con payload randomico, 1MB di dati e un delay durante l'invio >

CONCLUSIONI SULL'IMPLEMENTAZIONE DEI COVERT CHANNEL

In queste diapositive si è quindi presentato lo sviluppo e l'implementazione di un canale nascosto tramite il protocollo ICMP. L'obiettivo era dimostrare come il protocollo, fondamentale per il funzionamento delle reti, potesse essere sfruttabile per la creazione di un Covert Channel che possa esfiltrare delle informazioni.

Si è descritto l'implementazione della comunicazione fra le entità presenti nel sistema e successivamente l'implementazione delle seguenti tipologie di Covert Channel:

- Storage Covert Channel
implementato inserendo i dati nei campi dei messaggi ICMP.
- Timing Covert Channel
implementato codificando le informazioni tramite gli intervalli temporali definiti dalla funzione $\text{min_delay} + (\text{byte}/255) * (\text{max_delay} - \text{min_delay})$ al cui valore finale verrà aggiunto del rumore casuale.
- Behavioural Covert Channel
implementato codifica i dati attraverso l'invio degli eventi definiti dalla coppia Tipologia e Codice del messaggio ICMP.

Dopodiché si sono testati i Covert Channel tramite RITA. Nei test in cui si invia una singola volta i dati RITA non ha rilevato la presenza di un Covert Channel. Si è quindi proceduto ad eseguire test

che inviavano più volte i dati. I risultati ottenuti sono stati che l'utilizzo di un delay fra l'invio di un blocco di dati ed il successivo permette di abbassare la visibilità del Covert Channel. Anche la falsificazione dei mittenti permette questa cosa, a patto che gli indirizzi IP utilizzabili non vengano utilizzati in sequenza per ogni pacchetto inviato ma a blocchi. Inoltre anche un tempo di riposo fra sessioni di invio dati permette la cosa.

In sintesi la rilevabilità di un Covert channel dipende dai seguenti fattori:

- La frequenza ~~di trasmissione~~ con cui si inviano i dati.
- Quanti pacchetti vengono mandati da una sorgente in uno o più intervalli di tempo.

Le conclusioni sono quindi che sebbene sia un protocollo funzionale al monitoraggio di rete, i messaggi ICMP devono essere limitati solo a quelle tipologie essenziali alla rete che sono la tipologia Echo Request/Reply (usata in strumenti come ping), la tipologia Destination Unreachable e la tipologia Time Exceeded. Il resto delle tipologie o sono deprecate o non necessarie al monitoraggio della rete. E fra questi bisogna monitorare il loro utilizzo ed il loro contenuto siccome un messaggio Echo Request può contenere dati nel campo payload o può essere inviato secondo specifici pattern temporali. Ciò richiederà un'ispezione dei pacchetti presenti nel flusso, un'analisi statistica per la ricerca di pattern e la limitazione nella rete dei messaggi ICMP.